ROIP Module User Guide

Nov 4, 2026 · @George

Overview

A pair of 551 ROIP modules links two radio sites over any IP network, so a radio at one site can be heard and keyed from the other. This guide covers firmware v85.

Each module sits between local radio equipment and an Ethernet connection. Modules work in pairs: whatever goes in at one site comes out at the other, in both directions at once (full duplex).

A pair carries four things between sites:

- **Audio**: the local audio goes to the far site, and the far site's audio is played out locally.
- **PTT and COR**: when local equipment asserts PTT, audio is sent; when audio arrives from the far site, the module asserts COR.
- **DCN serial data**: an RS-485 bus at one site is bridged byte for byte to the RS-485 bus at the other.
- **Two logic lines**: two inputs at one site drive two outputs at the other.

Setup is done with short text commands over the USB console, with the guided `//setup` command, or from a phone over the module's own Wi-Fi setup network. Settings are saved in the module and survive a power cycle.

What's new since v66

Version	Change
v85	Opus in 20 ms packets between v85 modules (//opusframe) and forward error correction (//opusfec); a single lost Opus packet is rebuilt from the next one
v84	Follows the remote's address when it dials in with the correct password, and its NAT ports; adaptive jitter buffer, 60-100 ms (//jitter)

Version	Change
v83	//looptest echo test; LINK CHECK and LEVELS display pages; //levels; audio and codec questions in //setup and on the Wi-Fi page; QR code to join Wi-Fi setup
v82	//linktest and //report; public-address check in //status; DCN keepalive; watchdog and restart reason; 64-bit traffic counters
v81	DATA RATE display page: meters for data in and out, total rate and data volume
v80	//ping internet test; DCN command processor: frames sent to the module's //setaddr address run on the module, with PING, ROLLCALL, STATUS and NETPING replies
v79	`//status` and `//info` give a one-line-per-item summary; the per-direction table moved to `//status,detail`
v78	Emphasis set separately for each path: `//audioin,flat|pre|de` and `//audioout,flat|pre|de` (replaces the v77 flat/radio presets)
v77	Flat/radio presets for the radio-connection filters (replaced in v78)
v76	Software de-emphasis and pre-emphasis (750 µs), 300 Hz input high-pass, 3 kHz output low-pass; `//bitrate` data-rate report
v75	Display sends only the changed parts of the screen, so serial traffic no longer breaks up audio
v74	Unreachable-gateway guard; `//dacprime`; `//rs485hw`; worst-case timing figures in `//net`; DHCP renews during very long transmissions
v73	DCN dialing backs off when the peer can't be reached; quieter console at boot

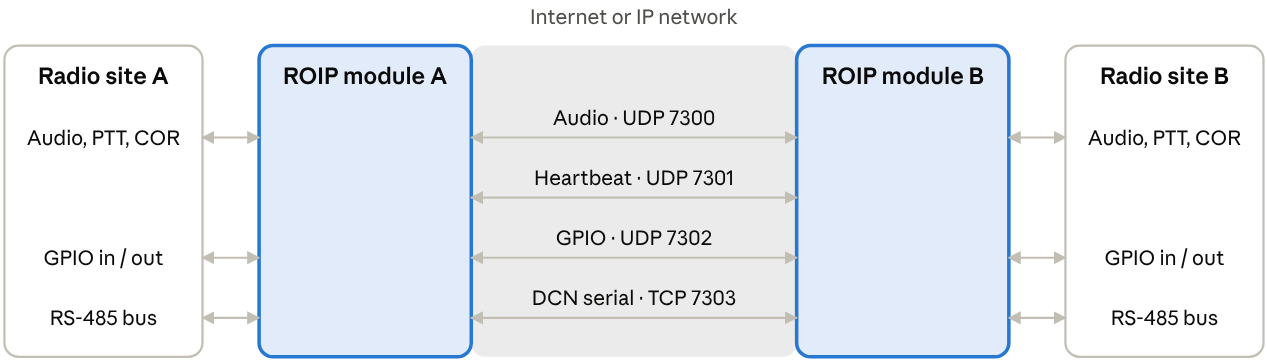
Version	Change
v72	Wi-Fi setup mode; link check in both directions; boot summary; clock-drift correction; faster, sequenced GPIO; GPIO force commands
v71	Factory default is a static address, 192.168.1.73
v70	`//setup` guided setup; `//net` regrouped, with the command for each setting
v69	Start-up splash screen
v68	Command history (Up/Down arrows, `//history`); ADPCM level fix
v67	Peer can be a DNS name (dynamic DNS); `//dns`; new default ports 7300-7303 and password `sierra`

If you are upgrading from v66 or earlier, both modules must be updated together: the default ports and password changed in v67.

How it works

Each module sends four independent streams to its peer, each on its own port, so one stream can fail without taking down the others.

Each module sends four independent streams to its peer



two-site link · four streams with default ports

Audio, GPIO and DCN carry the local equipment's signals; the heartbeat runs module to module only.

Stream	Protocol	Default port	Sent when	Purpose
Audio	UDP	7300	PTT active (plus a keepalive every 2 s when idle)	10 ms audio frames, 16 kHz mono (Opus between v85 modules: 20 ms packets)
Heartbeat	UDP	7301	Every 1 s	Tells each side the other is reachable; carries loss figures, which streams it is receiving, its firmware version, and the addresses each side sees (v82)
GPIO	UDP	7302	At once on a change, repeated after 20 and 50 ms, plus every 150 ms	State of the two logic inputs
DCN serial	TCP	7303	When RS-485 data arrives	Password-protected byte-for-byte serial bridge

Audio and PTT/COR

1. Local equipment pulls **PTT_INPUT** low (J3 pin 4).
2. The module captures audio, applies any input filters set, encodes it, and sends a numbered 10 ms frame every 10 ms.
3. The peer buffers about 60 ms of frames to smooth out network jitter, then plays them out of its audio output.
4. While frames are arriving, the peer asserts **COR_OUTPUT** (J3 pin 2, active low).
5. When PTT is released, the sending module can play a courtesy tone, then sends an end marker so the peer drops COR at once. If the end marker is lost, COR drops after the audio timeout (300 ms) plus the COR tail (500 ms).

Both directions work at the same time: a module can send while it receives.

Jitter buffer and clock drift

The receiving module plays frames in sequence-number order, not arrival order:

- A lost frame is replaced with silence. With Opus, a single lost packet is first rebuilt from the next packet (see Audio codecs).
- Frames that arrive out of order are put back in order.
- If the stream pauses, the buffer refills to its target depth before resuming rather than playing stale audio.

The target depth starts at 60 ms. Since v84 it adapts: when two mid-stream underruns happen within a minute, the network's jitter is larger than the cushion, so the target grows by 10 ms, up to 100 ms. After 10 minutes with no underruns it gives 10 ms back, down to 60 ms, so a calm link keeps the original delay. Each step up takes effect at the re-buffer the underrun has already started, so it adds no glitch of its own. `//jitter` shows the target; `//jitter,3` to `//jitter,10` fixes it in 10 ms frames, and `//jitter,auto` returns to adapting.

The two modules run on separate crystals, so their sample clocks differ by up to about 40 ppm, one 10 ms frame every few minutes. Since v72 the receiver resamples by up to ±500 ppm to keep its buffer at the target depth. That is a pitch change of under 1 cent, inaudible, and it removes the periodic 10 ms skip or gap on long transmissions.

The module also keeps 30 ms of audio queued in its output (`//dacprime`), so a brief stall at the module itself doesn't interrupt playback.

Audio codecs

The sending module picks the codec with `//codec` . Every packet names its codec and the receiver decodes whatever arrives, so the two ends don't have to match. To change both directions, set it on both modules. Opus is the only codec with an adjustable rate.

Codec	Audio data	Per direction on the internet (IP)	Quality	CPU load	Suits
PCM (default)	256 kbps	282 kbps	Uncompressed, transparent	Lowest	A LAN, or fibre/cable with plenty of upload
u-law (G.711)	128 kbps	154 kbps	Telephone quality, 8-bit companding	Very low	Upload of 0.5-1 Mbps

Codec	Audio data	Per direction on the internet (IP)	Quality	CPU load	Suits
ADPCM (IMA, 4:1)	66 kbps	93 kbps	Close to PCM for voice; each packet carries its decoder state, so a lost packet doesn't upset the next	Very low	Upload of 200-500 kbps
Opus	6-64 kbps, set by `//opusbitrate`	19-77 kbps in 20 ms packets, 33-90 kbps in 10 ms	Best quality per bit; a single lost packet is rebuilt from the next one	Highest: about 70% of one core	Limited upload, lossy links

The internet figures are higher than the audio data because every packet also carries 5 bytes of codec type and sequence number plus 28 bytes of UDP/IP headers. At 100 packets a second (10 ms packets) that is 26.4 kbps per direction before any audio; at 50 a second (20 ms Opus packets) it is 13.2 kbps. The figures apply while audio flows in that direction; full duplex doubles the total only while both ends talk. An idle link uses about 0.5 kbps for the heartbeat, GPIO refresh and keepalive. `//bitrate` shows the measured rates, and `//net` shows the codec being sent and the one being received.

Opus bitrate and audio bandwidth

`//opusbitrate` sets the target rate of this module's Opus encoder, 6000-64000 bps, default 64000. It changes only the audio this module sends, only while `//codec,opus` is set, applies at once and is saved. As the rate drops, Opus narrows the audio band to keep voice clear:

`//opusbitrate`	Opus audio band	Internet, 20 ms packets (v85 default)	Internet, 10 ms packets	Sound
64000 (default)	Wideband, up to 8 kHz	77 kbps	90 kbps	Transparent
48000	Wideband, 8 kHz	61 kbps	74 kbps	Transparent
32000	Wideband, 8 kHz	45 kbps	58 kbps	Transparent for voice

`//opusbitrate`	Opus audio band	Internet, 20 ms packets (v85 default)	Internet, 10 ms packets	Sound
16000	Wideband, 8 kHz	29 kbps	42 kbps	Very good voice
12000	Mediumband, 6 kHz	25 kbps	38 kbps	Good voice
8000	Narrowband, 4 kHz	21 kbps	34 kbps	Usable, slightly rough
6000 (minimum)	Narrowband, 4 kHz	19 kbps	33 kbps	Rough but intelligible

The module samples at 16 kHz, so 8 kHz is the widest band Opus can carry. The switch points between bands are the Opus library's automatic choices for voice and are approximate. FM radio audio already stops at about 3 kHz, and `//txlpf,on` limits the output to 3 kHz, so the narrower bands lose little; the roughness at 6-8 kbps comes from coarser coding. The other Opus settings are fixed: 16 kHz mono, voice mode, complexity 1.

Opus packet length and error correction

Since v85, Opus has two more settings, both on by default and saved:

- **`//opusframe,20`** sends one packet per 20 ms instead of per 10 ms. That halves the packets and the header overhead, so 16 kbps Opus drops from about 42 to about 29 kbps on the internet, for 10 ms more delay. A module sends 20 ms packets only while the remote's heartbeat says it runs v85 or later; otherwise it sends 10 ms, so a v85 module still works with an older one. `//opusframe,10` always sends 10 ms.
- **`//opusfec,on`** adds forward error correction: each packet also carries a low-rate copy of the one before. Opus uses it in its speech mode, at voice bitrates. `//opusfec,off` saves those bits.

Receiving needs no setting. A module decodes 10 ms and 20 ms packets alike, and when exactly one packet is missing it rebuilds it from the next packet before playing on: from that packet's FEC copy if it has one, otherwise by Opus's own concealment. Either is far less noticeable than the silence other codecs use. `//net` counts the frames rebuilt this way.

Adaptive Opus bitrate

`//adaptive,on` lets the module choose its own Opus rate from the loss the peer reports in its heartbeat. It steps one tier at a time:

Tier	Opus rate	Moves to this tier when
Good	64000	The peer reports 1% loss or less for 5 reports in a row (about 10 s)
Degraded	48000	Stepping up from Poor, or down from Good
Poor	32000	The peer reports 3% loss or more for 2 reports in a row (about 4 s)

Between 1% and 3% loss the rate holds. Adaptive mode overrides `//opusbitrate`, and turning it off sets 64000. It never goes below 32 kbps, so on a slow link turn it off and set the rate by hand.

Choosing a codec

Each site's upload speed carries what its own module sends. Leave about twice the internet figure as headroom.

Upload at the sending site	Use
More than 1 Mbps, or a LAN	`//codec,pcm`
300 kbps to 1 Mbps	`//codec,ulaw` or `//codec,adpcm`
100-300 kbps, or noticeable packet loss	`//codec,opus` with `//opusbitrate,16000` to `32000`
Under 100 kbps	`//codec,opus` with `//opusbitrate,8000` to `12000`, `//adaptive,off`

For the lowest rate, type these on both modules:

```
//adaptive,off
//codec,opus
//opusbitrate,6000
```

That is about 19 kbps per direction between v85 modules. `//opusbitrate,64000` returns to the default rate. `//codec,opus` is also a choice in `//setup` and on the Wi-Fi setup page.

DCN serial bridge

Bytes arriving on the local RS-485 bus are sent to the peer, and bytes from the peer are written out on the local bus. The bridge is transparent unless the module is given a DCN address (see the next section).

Both modules listen for a connection and also dial out, so either side can establish the link. The side that dials sends the shared password; the side that accepts checks it and drops the connection if it doesn't match. No data crosses until the password is accepted.

If dialing fails, the module waits twice as long before each new attempt, up to 60 s, and stops dialing while audio is flowing. It keeps listening the whole time, so the peer can still dial in.

Since v82 the module turns on TCP keepalive for the link. After 10 s with no data it checks the far end is still there; if the far end has gone (power cut, a changed address, a router dropping its state) the module closes the link within about two minutes and redials, instead of showing a dead link as connected.

DCN command processor

Since v80 a module can be addressed on the DCN bus like the other Sierra Radio Systems modules. `//setaddr,25` gives it address 25 (1-99, saved); `//setaddr,0`, the default, turns this off and leaves the bridge fully transparent.

DCN frames have the family format `/FFTT:PAYLOAD:XX` followed by CR LF, where FF is the sender's address, TT the destination, and PAYLOAD a command with comma-separated fields. With an address set, the module watches both directions of the bridge, the local RS-485 bus and the data from the peer:

- A frame whose TT is this module's address is taken out of the stream, not forwarded, and run on the module. The reply goes back the way the frame came, addressed to FF: `/25FF:<reply>:XX`.
- Everything else passes through unchanged. Only the first six characters of a frame, `/FFTT:`, are held while the address is checked.

A controller at either site can therefore command either module: give the two modules different addresses. Address 00 is left for the controller.

Payload	Reply
`PING`	`PING,ROIP1,<MAC>,ONLINE`
`ROLLCALL`	The same as PING, after the module's address × 200 ms, for bus discovery
`STATUS`	`STATUS,ROIP1,eth,peer,audio,serial,gpio,ptt,cor,in1,in2,out1,out2`, each 1 or 0; audio and gpio are 1 only when working in both directions
`NETPING[,host[,n]]`	Pings host (default 8.8.8.8, 4 times) as `//ping` does: first `NETPING,ROIP1,STARTED,<ip>`, then `NETPING,ROIP1,<ip>,<sent>,<received>,<avg ms>,OK` or `FAIL`
A console command without `//`	Runs it if it is on the allowed list, for example `RXLEVEL,60` runs `//rxlevel,60`. Reply: `RXLEVEL,ROIP1,OK,<first line of the console reply>`, or `ERR`

Allowed over DCN: `info`, `status`, `version`, `filters`, `bitrate`, the audio level, codec and filter commands, `tone`, the courtesy-tone and PTT timing commands, `dacprime`, `driftcorrect`, the GPIO force commands, `display`, `scopegain`, `faultlog`, `timingreset` and `diag`. Everything that changes addressing, ports, the link password, the DCN baud rate or RS-485 mode, or reboots or erases the module is refused with `ERR,NOT ALLOWED`, and so is `//net`, because it shows the link password. Refused frames are logged on the USB console.

Frames can also be typed by hand in a terminal on the bus: the module waits up to 1.5 s for the `/FFTT:` header and 3 s between later characters. There is no line editing.

GPIO pass-through

Input 1 at one site drives output 1 at the other, and input 2 drives output 2. Inputs are active low. Each change carries a sequence number, so a delayed packet can't flip an output back to an old state. If no update arrives from the peer for 2 s, the outputs return to idle rather than holding their last state.

Peer address and the gateway check

The peer's address can be an IP address or a DNS name, such as a dynamic DNS name. A name is looked up at start-up and again every 5 minutes while audio is idle, or every 30 s while the peer can't be heard. If the address changes, the module follows it.

If the module's own address, netmask or gateway is wrong for the site, nothing it sends can leave the local network. Since v74 the module detects this, pauses all traffic to the peer, says so on the console and the display (`NO GW`), and retries every 10 s. This keeps the console and display responsive on a misconfigured module.

Address check (v82). Each heartbeat also carries the address and port the sender sees the other module's heartbeats arriving from. So each module knows its own site's public address, shown in `//status` as "This site's public", and notices when the remote's packets come from a different address than the one it is sending to. `//status` then prints a WARNING naming both addresses: the far site's public address has changed, or its DNS name is out of date.

Following the remote (v84). When the remote dials in on the DCN link and its password is accepted, the address it came from is certainly the remote's. If it differs from `//remoteip` or its DNS name, audio, heartbeat and GPIO follow it at once, and `//status` shows "Remote address: following ...". It lasts until a reboot, a new `//remoteip`, or the DNS name changing to a new address. Nothing unauthenticated can move the address.

The module also follows the remote's NAT ports: when a stream's packets arrive from the remote's address with a different source port, because its router rewrote it, replies go back to that port. With both features, only one site strictly needs port forwards, as long as the other site can dial out.

Security

The DCN password is sent in clear text, and the audio, heartbeat and GPIO streams are not authenticated. Over the open internet, run the pair over a VPN, or restrict the forwarded ports to the other site's address. Anything on the RS-485 bus can send DCN commands to a module with an address set; only the allowed list runs. A module only changes where it sends after the remote has proven the link password, and `//looptest` test packets are only sent to a remote running v83 or later.

Getting started: a pair over the internet

Each module needs a fixed address on its own network, the other site's public address, four forwarded ports on each router, and the same link password. Five answers in `//setup`, or on the Wi-Fi setup page, cover everything but the router.

Before you start

- **A reachable public address at each site.** A static public IP or a dynamic DNS name (for example `parc1.mywire.org`) both work. A site behind carrier-grade NAT (router WAN address 100.64.x.x to 100.127.x.x) can't accept incoming traffic: use a VPN instead.
- **Upload bandwidth.** With the default PCM codec each direction uses about 300 kbps. On slower connections, choose u-law or ADPCM, or Opus at about 30 kbps.
- **A way to configure the module.** Either a USB cable and a terminal program at 115200 baud with local echo on, or a phone or laptop with Wi-Fi.
- **Test on the bench first.** Configure both modules, connect them to the same switch with LAN addresses, and confirm they link before taking them to the sites.

Example addresses

The steps below use these example addresses. Substitute your own.

Setting	Site A	Site B
Public address (router WAN or DNS name)	203.0.113.10	siteb.mywire.org
Module LAN address	192.168.1.50	192.168.10.50
Netmask	255.255.255.0	255.255.255.0
Gateway (router LAN)	192.168.1.1	192.168.10.1
Remote site entered on this module	siteb.mywire.org	203.0.113.10

Step 1: configure each module

Use either method on each module. Both ask the same things: static or DHCP, the address details, the remote site, the link password, and since v83 the audio input and output emphasis and the codec (with the Opus bitrate when Opus is chosen).

Option A: guided setup over USB. Type `//setup` and answer each question. Press Enter to keep the value in brackets, or type `//cancel` to quit without changes:

```
> //setup
Use static IP or DHCP? S/D (Default: STATIC): s
What is your device's static IP address? (Default: 192.168.1.73):
192.168.1.50
Enter the device netmask (Default: 255.255.255.0):
Enter the network gateway (Default: 192.168.1.1):
Enter the remote device WAN address, IP or DNS name (required):
siteb.mywire.org
Enter the link password (Default: sierra): YourSharedSecret
Audio input emphasis: F = flat, D = de-emphasis (input from a radio's
discriminator),
  P = pre-emphasis. F/D/P (Default: FLAT):
Audio output emphasis: F = flat, P = pre-emphasis (output to a radio's
modulator),
  D = de-emphasis. F/P/D (Default: FLAT):
Codec for the audio this board sends (per direction on the internet):
  PCM 282 kbps, ULAW 154 kbps, ADPCM 93 kbps, OPUS 33-90 kbps.
  PCM/ULAW/ADPCM/OPUS (Default: PCM): opus
Opus bitrate, 6000-64000 bps; 16000 is very good voice at ~42 kbps (Default:
64000): 16000
--- Setup summary ---
...
Save these settings? Y/N (Default: Y): y
Reboot now so the network settings take effect? Y/N (Default: Y): y
```

Choosing de-emphasis on the input also turns on the 300 Hz high-pass, and pre-emphasis on the output turns on the 3 kHz low-pass, as recommended in Connecting to radios. The rates quoted in the questions are for 10 ms packets; between v85 modules Opus uses about 13 kbps less.

Option B: Wi-Fi setup from a phone.

1. Hold the mode button while powering up, and keep holding it for 2 s.
2. The display shows a QR code beside the Wi-Fi network name (SRS-ROIP-xxxx) and its password. Point the phone camera at the QR code to join, or type the name and password. The password is unique to the module and only shown on the module itself.
3. Once joined, the setup page opens by itself; if it doesn't, browse to `http://192.168.4.1`.
4. Fill in the form, including the Audio section, and tap **Save and restart**.

The module returns to normal operation by itself after 10 minutes with no activity on the page, or when the mode button is pressed again. Wi-Fi is never on in normal operation.

If you'd rather use DHCP, choose it and add a DHCP reservation in the router for the module's MAC address. The boot summary and `//net` show the MAC.

Step 2: forward ports on each router

Forward these to the module's LAN address:

Protocol	Port	Stream
UDP	7300	Audio
UDP	7301	Heartbeat
UDP	7302	GPIO
TCP	7303	DCN serial

Limit each rule to the other site's public address if your router supports it. The DCN link needs TCP 7303 forwarded at only one of the two sites, but forwarding it at both is simplest. The UDP ports must match on both modules: if you change one, change it on both.

Step 3: optional settings

- **Codec**: `//codec,opus` with `//opusbitrate,16000` (or `//codec,ulaw`, `//codec,adpcm`) for limited upload bandwidth. See Choosing a codec.
- **DCN baud rate**: `//dcnbaud,9600` to match your RS-485 bus.
- **Emphasis**: `//audioin` and `//audioout` if the module is wired to a radio's discriminator or modulator, or to equipment that expects one. See Connecting to radios.

Step 4: check the link

About 10 s after start-up, each module prints a status summary on the console. Type `//status` (or `//info`) at any time for the same summary. A working pair looks like this:

```
> //status
--- Status ---
  My IP address      192.168.1.50  (static)
  Ethernet           connected
  Remote IP address  siteb.mywire.org  (198.51.100.20)
  Remote device      connected
  Audio channel      working
  Serial channel     working
  GPIO channel       working
--------------
```

Audio and GPIO show `working` only when they work in both directions.

When an item isn't working, its line says why, and `//status,detail` shows each stream in each direction with a hint on which site's router to check. See the example under Examples.

If anything isn't working, run `//linktest`. It checks Ethernet, this module's address, the gateway, the internet, the remote's DNS name and router, then each stream in each direction, and prints PASS or FAIL with what to fix. Then `//looptest` measures loss each way, round-trip time and jitter on each stream (both modules need v83 or later). See Examples.

The Link LED should be on solid at both sites. To test audio end to end, send a tone from one site and listen at the other:

```
//tone,1000,3000
```

Then set levels to suit your equipment with `//rxlevel` (audio sent to the far site) and `//txlevel` (audio played out locally).

Connecting to radios: emphasis and filters

Each audio path has its own emphasis setting, so the module can match whatever is wired to it at each end. Both paths are flat by default.

Setting	Input path (audio in → network)	Output path (network → audio out)
`flat`	Passed through unchanged	Passed through unchanged

Setting	Input path (audio in → network)	Output path (network → audio out)
`de`	From a discriminator: de-emphasized before sending	Network audio is pre-emphasized; equipment here wants flat audio
`pre`	From a flat source: pre-emphasized before sending	To a modulator: pre-emphasized

Set them with `//audioin,flat|pre|de` and `//audioout,flat|pre|de`. Two band-limiting filters are switched on their own:

Filter	Path	What it does
`//rxhpf,on`	Input, before the emphasis	300 Hz high-pass, 4th order. Removes CTCSS, DCS and DC offset
`//txlpf,on`	Output, after the emphasis	3 kHz low-pass, 4th order. Keeps out-of-band energy off a modulator

`//filters` shows all four settings at once.

Common setups

Wired to	Commands
Speaker or line audio at both ends	`//audioin,flat`, `//audioout,flat` (the defaults)
A radio's discriminator and modulator	`//audioin,de`, `//audioout,pre`, `//rxhpf,on`, `//txlpf,on`
A repeater controller that expects discriminator and modulator connections	Same as a radio: the module's output then looks like discriminator audio and its input accepts modulator audio
A discriminator here, a speaker-level radio at the far end	Here: `//audioin,de`, `//rxhpf,on`. Far end: flat

Time constant

All emphasis uses a 750 µs time constant, the standard for land-mobile and amateur FM (TIA-603: 6 dB per octave from 300 to 3000 Hz). The measured response stays within about 1 dB of ideal across that band. Change it with `//emphtc,<µs>` only if your equipment uses another standard; 75 µs is the broadcast-FM value.

Levels

De-emphasis is unity at 1 kHz, so the level at 1 kHz doesn't change. Pre-emphasis is unity at 3 kHz so it can never clip a full-scale signal, which means 1 kHz comes out about 9 dB lower.

- After setting `pre` on a path, raise that path's level by about 14 steps: `//rxlevel` for the input, `//txlevel` for the output (for example 50 → 64). Each step above 50 adds 0.64 dB.
- The console prints the change and which level to adjust whenever you switch. It never changes a level itself.
- Set deviation with a 1 kHz test tone sent from the far site (`//tone,1000,10000`) and a deviation meter on the transmitter.

Limits

Input de-emphasis boosts a 100 Hz tone about 13 dB relative to 1 kHz, so use `//rxhpf,on` with it. The 300 Hz high-pass gives −52 dB at 67 Hz, −38 dB at 100 Hz and −24 dB at 150 Hz, but only −7 dB at 254 Hz, so the highest CTCSS tones may still be faintly audible. Test tones and courtesy tones are generated after the input filters, so they are sent flat.

Command reference

Every command starts with `//`, is typed on the USB console at 115200 baud, and is not case-sensitive (except the password itself). **Saved** settings survive a reboot; **Reboot** means the change takes effect after `//reboot`. The Up and Down arrows recall the last 50 commands.

Setup and status

Command	What it does	Default
`//setup`	Guided setup: addressing, remote site, link password	
`//status` (or `//info`)	Summary: my IP, Ethernet, remote IP, remote device connected, and whether the audio, serial and GPIO channels work	
`//status,detail`	Each stream in each direction, with hints on which router to check	
`//net` (or `//config`)	Every setting, grouped, with the command that changes it, plus live status	
`//version`	Firmware version	
`//help`	Lists the commands	
`//history`	Lists recent commands	
`//reboot`	Restarts the module; settings are kept	
`//factoryreset`	Explains the reset; then `//factoryreset,confirm` erases every setting and reboots	
//linktest	Checks the link step by step (Ethernet, address, gateway, internet, DNS, remote router, each stream each way, serial) with PASS / FAIL / WARN and what to fix. Takes about 10 s; audio keeps working	
//report	Everything support needs in one block to copy and email: version, uptime, restart reason, status, settings (password hidden), data rates, filters, fault log. Audio pauses for about 1 s while it prints	

Link

Command	What it does	Default
`//remoteip,<ip or name>`	Peer site's public address, or a DNS name. Saved	not set

Command	What it does	Default
`//setpassword,<text>`	Shared DCN link password; must match the peer exactly. Saved	sierra
`//remoteport,<1-65535>`	Peer's DCN TCP port. Saved	7303
`//localport,<1-65535>`	This module's DCN TCP listening port. Saved, applies at once	7303
`//connect`	Looks up a DNS name again, then drops and re-dials the DCN link at once	
`//reconnectinterval,<ms>`	Base time between DCN dial attempts (100-60000); doubles after each failure, up to 60 s. Saved	2000

Network

Command	What it does	Default
`//dhcp,on` / `off`	Uses DHCP, or the static address only. Saved, reboot	off
`//staticip,<ip>`	Static address; also the fallback if DHCP fails. Saved, reboot	192.168.1.73
`//netmask,<mask>`	Static netmask. Saved, reboot	255.255.255.0
`//gateway,<ip>`	Static gateway. Saved, reboot	192.168.1.1
`//dns,<ip>` / `auto`	DNS server for name lookups; `auto` uses the DHCP server's, or the gateway. `//dns` shows it. Saved	auto
`//mac,<aa:bb:cc:dd:ee:ff>` / `auto`	Overrides the MAC address, or returns to the chip-derived one. Saved, reboot	unique per chip
`//audioport,<n>`	Audio UDP port; must match the peer. Saved	7300
`//heartbeatport,<n>`	Heartbeat UDP port; must match the peer. Saved	7301

Command	What it does	Default
`//gpioport,<n>`	GPIO UDP port; must match the peer. Saved	7302
`//hbinterval,<ms>`	How often a heartbeat is sent (100-60000). Saved	1000
`//hbtimeout,<ms>`	Silence before the peer counts as lost (100-120000). Saved	3000

Audio

Command	What it does	Default
`//rxlevel,<0-100>`	Level of local audio sent to the peer (50 = unity, 0.64 dB per step above). Saved	50
`//txlevel,<0-100>`	Level of peer audio played out locally (50 = unity). Saved	50
`//inputlevel,<0-30>`	Analog input gain in dB, in 3 dB steps. Saved	0
`//codec,pcm` / `ulaw` / `adpcm` / `opus`	Codec for the audio this module sends: 282, 154, 93 or 33-90 kbps per direction on the internet. Saved	pcm
`//opusbitrate,<6000-64000>`	Target rate of the Opus audio this module sends, in bits per second; below about 14000 Opus narrows the audio band (see Audio codecs). Used only with //codec,opus. Saved	64000
`//adaptive,on` / `off`	Steps the Opus rate between 64000, 48000 and 32000 from the loss the peer reports; overrides //opusbitrate, and off sets 64000. Saved	off
`//tone,<hz>,<ms>[,<level>]`	Sends a test tone to the peer (100-4000 Hz, up to 300000 ms, level 0-100)	
`//dacprime,<3-8>`	10 ms frames kept queued in the audio output: more rides out longer stalls but adds delay. Saved	3

Command	What it does	Default
`//driftcorrect,on` / `off`	Clock-drift correction on playback; `//driftcorrect` shows the current correction. Not saved	on
//opusframe,10 / 20	Opus packet length. 20 halves the header overhead for 10 ms more delay; used only while the remote runs v85 or later, else 10. Saved	20
//opusfec,on / off	Opus forward error correction: each packet carries a copy of the one before, so a single lost packet is rebuilt. Saved	on
//jitter,auto / 3-10	Receive buffer: auto grows from 60 ms to 100 ms on repeated underruns and shrinks back after 10 quiet minutes; a number fixes it in 10 ms frames. //jitter shows it. Saved	auto

Emphasis and filters

Command	What it does	Default
`//audioin,flat` / `pre` / `de`	Emphasis on audio in → network; `de` for a discriminator. `//audioin` shows it. Saved	flat
`//audioout,flat` / `pre` / `de`	Emphasis on network → audio out; `pre` for a modulator. `//audioout` shows it. Saved	flat
`//rxhpf,on` / `off`	300 Hz high-pass on the input. Saved	off
`//txlpf,on` / `off`	3 kHz low-pass on the output. Saved	off
`//emphtc,<25-2000>`	Emphasis time constant in µs. Saved	750
`//filters`	Shows all four settings and any level change to make	

PTT, COR and courtesy tone

Command	What it does	Default
`//audiotimeout,<ms>`	Time after the last audio frame before COR starts to drop (0-60000). Saved	300
`//pttdelay,<ms>`	COR hang time after that (0-60000). Saved	500
`//unkeytone,<hz>,<ms>[,<level>]`	Courtesy tone sent when PTT is released; `<ms>` = 0 turns it off. Saved	off
`//unkeybeeps,<1-3>`	Number of beeps in the courtesy tone. Saved	1
`//unkeybeepgap,<ms>`	Silence between beeps (0-5000). Saved	100
`//unkeypredelay,<ms>`	Silence before the tone (0-10000). Saved	200
`//unkeypostdelay,<ms>`	Silence after the tone (0-10000). Saved	200

DCN serial and GPIO

Command	What it does	Default
`//dcnbaud,<300-921600>`	RS-485 baud rate; must match the bus. Saved, applies at once	9600
`//rs485hw,on` / `off`	Lets the UART switch the RS-485 driver itself instead of the firmware waiting for each send. Check with a scope before relying on it. Saved, reboot	off
`//localgpio1,on` / `off`, `//localgpio2,...`	Forces this module's output 1 or 2, ignoring the peer. Not saved	
`//remotegpio1,on` / `off`, `//remotegpio2,...`	Forces what this module sends for input 1 or 2, so the peer's output follows. Not saved	
`//gpiorelease`	Releases every forced GPIO	

Command	What it does	Default
//setaddr,<1-99> / 0	This module's DCN address: frames /FFTT:...:XX with TT = it are run here, not forwarded; 0 = bridge fully transparent. //setaddr shows it. Saved	0 (off)

Diagnostics

Command	What it does	Default
//ping[,<ip or name>[,<1-20>]]	Pings an address (default 8.8.8.8, 4 requests 1 s apart) and says what the result means: internet OK, gateway not answering, or no route past the router. Runs in the background; //ping,stop ends it	
`//bitrate`	Data rates per stream, sent and received: payload, IP, Ethernet and wire kbps, plus the expected audio rate for the current codec	
`//bitrate,on` / `off` / `reset`	One `[RATE]` line per second, stop, or clear the averages	off
`//console,on` / `off`	Shows or hides background log messages; command replies always show	off
`//diag,on` / `off`	PTT, capture and jitter-buffer statistics in the log	off
`//instrumentation,on` / `off`	Timing and packet diagnostics in the log	off
`//faultlog,on` / `off` / `clear`	Starts, stops or empties the fault log; `//faultlog` prints it	off
`//timingreset`	Clears the worst-case timing figures shown in `//net`	
`//display,on` / `off`	Turns all display updates on or off	on

Command	What it does	Default
`//scopegain,in` / `out` / `both,<1-16 or auto>`	Scope page vertical gain; `//scopegain` shows it. Saved	2
//looptest	Echo test: 20 test packets on each UDP stream to the remote and back; shows how many reached the remote and came back, round trip and jitter. Needs v83 or later at both ends	
//levels	Peak input and output level in dBFS over the last second, with clipping; aim for -6 to -3 dBFS on speech	
//levels,on / off	One [LEVEL] line per second; measures the input even without PTT	off

The switches marked Not saved, and the console, diagnostic, fault-log and display switches, return to their defaults on every boot.

Examples

These console sessions show what you type after the `>` prompt and what the module replies. Long replies are shortened with `...`.

Find a missing port forward with //status

```
> //status
--- Status ---
  My IP address      192.168.1.50  (static)
  Ethernet           connected
  Remote IP address  siteb.mywire.org  (198.51.100.20)
  Remote device      connected
  Audio channel      NOT working -- the remote isn't receiving ours
  Serial channel     working
  GPIO channel       working
  //status,detail shows each direction and what to check.
--------------
> //status,detail
--- Link status ---
  This board   192.168.1.50  (Ethernet up)
```

```
  Remote site  siteb.mywire.org -> 198.51.100.20
  Channel      Port       Receiving from peer  Peer receiving us
  Heartbeat    UDP 7301   yes                  yes
  Audio        UDP 7300   yes                  NO
  GPIO         UDP 7302   yes                  yes
  DCN serial   TCP 7303   working (we dialed out)
  > The peer isn't receiving our audio -- check UDP 7300 is forwarded on the
PEER site's router.
-------------------
```

The summary shows audio isn't working, and the detail table shows why: this module hears everything from site B, but site B isn't receiving this module's audio. The fix is on site B's router: forward UDP 7300 to its module.

Find a fault step by step with //linktest

```
> //linktest
--- Link test (takes about 10 s; audio keeps working) ---
  [PASS] Ethernet link             up
  [PASS] This board's address      192.168.1.50 / 255.255.255.0, gateway
192.168.1.1 (static)
  [PASS] Gateway                   192.168.1.1 answers (1 ms)
  [PASS] Internet                  8.8.8.8 answers (18 ms)
  [PASS] Remote's DNS name         siteb.mywire.org -> 198.51.100.20
  [WARN] Remote site's router      198.51.100.20 doesn't answer ping
         -> Many routers ignore ping, so this alone isn't a fault.
  [PASS] Receiving from remote     heartbeat arriving
  [FAIL] Remote receiving us       nothing arriving there
         -> Forward UDP 7300-7302 to the remote board on the REMOTE site's
router.
  [FAIL] Audio (UDP 7300)          from remote: yes, to remote: NO
         -> Check UDP 7300 is forwarded on the router of the side that isn't
receiving.
  [FAIL] GPIO (UDP 7302)           from remote: yes, to remote: NO
         -> Check UDP 7302 is forwarded on the router of the side that isn't
receiving.
  [PASS] Serial (TCP 7303)         connected, we dialed out
--- 7 passed, 3 failed, 1 warning(s) ---
Fix first: Forward UDP 7300-7302 to the remote board on the REMOTE site's
router.
```

Everything at this site works, and site B's packets arrive here, but site B isn't receiving this module's UDP: site B's router is missing its forwards. Each step runs only if the steps before it passed, so the first FAIL is the one to fix.

Measure each stream with //looptest

```
> //looptest
Loop test: 20 test packets on each stream to the remote and back (about 3
s)...
--- Loop test: sent / reached remote / came back, round trip min/avg/max,
jitter ---
  Audio      UDP 7300  20 / 20 / 19   31/38/77 ms, jitter 6 ms
             lost 0 on the way there, 1 on the way back
  Heartbeat  UDP 7301  20 / 20 / 20   30/36/52 ms, jitter 4 ms
  GPIO       UDP 7302  20 / 20 / 20   31/37/49 ms, jitter 4 ms
Some test packets were lost -- see above.
```

The remote counts the test packets it receives, so a loss is placed on the way there or the way back. A stream with nothing coming back is blocked in at least one direction. Jitter well above about 30 ms is a sign to let `//jitter,auto` grow the buffer, or to fix it larger.

Set audio levels with //levels

```
> //levels
--- Audio levels, peak over the last second (aim for -6 to -3 dBFS on speech)
---
  Input  (audio in, after //inputlevel and //rxlevel):  -4.8 dBFS
  Output (to audio out, before //txlevel):              -11.2 dBFS
```

The input is measured while PTT is held, while the LEVELS page is showing, or with `//levels,on`, which prints a `[LEVEL]` line every second. CLIPPING after a figure means samples hit full scale: lower `//inputlevel` or `//rxlevel` for the input, or `//rxlevel` at the remote for the output. The LEVELS display page shows the same figures as bars.

Point a module at a dynamic DNS name

```
> //remoteip,siteb.mywire.org
Remote site set to hostname siteb.mywire.org
DNS: siteb.mywire.org -> 198.51.100.20
```

The module looks the name up again every 5 minutes, and sooner if the peer goes quiet, so it follows the far site when its address changes.

Connect to a radio's discriminator and modulator

```
> //audioin,de
Audio input emphasis: de
...
> //rxhpf,on
...
> //audioout,pre
Audio output emphasis: pre
Output level at 1 kHz is now 8.9 dB lower -- raise //txlevel by about 14 to
keep the same level.
...
> //txlpf,on
...
> //txlevel,64
TX audio level set to 64% (applied immediately)
> //filters
--- Audio filters ---
  Input  (audio in -> network):   emphasis de,  300 Hz high-pass ON
  Output (network -> audio out):  emphasis pre,  3 kHz low-pass ON
  Time constant: 750 us (land-mobile / amateur FM standard)
  Pre-emphasis is unity at 3 kHz, so 1 kHz is -8.9 dB on the output -- raise
//rxlevel or //txlevel to suit (~0.64 dB per step above 50).
```

Then send a 1 kHz tone from the far site and check deviation at the transmitter.

Check the data rate of a codec

Set the codec, hold PTT (or send a tone), and read the last second:

```
> //codec,ulaw
Audio codec set to ulaw
> //bitrate
--- Data rates ---
Last 1000 ms:
  Channel    Dir    pkt/s   payload        IP  Ethernet      wire   (kbps)
  Audio      out    100.0     132.0     154.4     168.8     184.8
  Audio      in       0.0       0.0       0.0       0.0       0.0
  Heartbeat  out      1.0       0.0       0.3       0.5       0.7
  Heartbeat  in       1.0       0.0       0.3       0.5       0.7
  GPIO       out      7.0       0.2       1.7       3.6       4.7
  GPIO       in       7.0       0.2       1.7       3.6       4.7
  DCN (est)  out      0.0       0.0       0.0       0.0       0.0
  DCN (est)  in       0.0       0.0       0.0       0.0       0.0
  Total      out    108.0     132.2     156.4     172.9     190.2
  Total      in       8.0       0.2       2.0       4.1       5.4
...
Expected while sending audio (u-law, 100 pkt/s): payload 132.0, IP 154.4,
Ethernet 168.8, wire 184.8 kbps
```

The measured audio row matches the expected figure. Use the IP column to compare against an internet plan's upload speed, and `//bitrate,on` to watch the rate change second by second.

Lower the data rate with Opus

On both modules:

```
> //adaptive,off
Adaptive Opus bitrate switching: OFF
> //codec,opus
Audio codec set to opus
> //opusbitrate,16000
Opus bitrate set to 16000 bps
```

Hold PTT and check `//bitrate`: the audio row should read about 42 kbps in the IP column at 100 packets a second, down from 282 kbps with PCM. `//opusbitrate,64000` goes back to the default.

Test the internet connection with //ping

```
> //ping
Pinging 8.8.8.8 via gateway 192.168.1.1, 4 requests 1 s apart. //ping,stop
ends it early.
Reply from 8.8.8.8: seq=1 time=18 ms
Reply from 8.8.8.8: seq=2 time=17 ms
Reply from 8.8.8.8: seq=3 time=22 ms
Reply from 8.8.8.8: seq=4 time=19 ms
--- 8.8.8.8: 4 sent, 4 received, 0% loss, round trip min/avg/max 17/19/22 ms
---
Result: internet connectivity OK.
```

The ping runs in the background, so audio and the console keep working. When the gateway setting is wrong for the site, each request reports `no ARP answer from the gateway`, and the result line says to check the IP address, netmask and gateway. When the gateway answers but nothing comes back, the problem is past the router. `//ping,siteb.mywire.org` tests the far site's router, though many routers don't answer pings.

Command a module over DCN

```
> //setaddr,25
DCN address: 25 -- frames /xx25:...:XX from the RS-485 bus or the network are
run here
and not forwarded; all other traffic passes through. Give the other board a
different address.
```

A controller at address 00 on either site's bus can then talk to it:

Sent on the bus	The module replies
`/0025:PING:XX`	`/2500:PING,ROIP1,02A1B2C3D4E5,ONLINE:XX`
`/0025:RXLEVEL,60:XX`	`/2500:RXLEVEL,ROIP1,OK,RX audio level set to 60% (applied immediately):XX`
`/0025:NETPING:XX`	`/2500:NETPING,ROIP1,STARTED,8.8.8.8:XX`, then about 4 s later `/2500:NETPING,ROIP1,8.8.8.8,4,4,19,OK:XX`
`/0025:REBOOT:XX`	`/2500:REBOOT,ROIP1,ERR,NOT ALLOWED:XX`

Each command and the module's console reply also show on the module's USB console.

Add a two-beep courtesy tone

```
> //unkeytone,1000,100,50
Unkey tone: 1000.00 Hz for 100 ms at 50% — will play automatically when
PTT_INPUT releases.
> //unkeybeeps,2
Unkey tone set to 2 beeps
Gap between beeps: 100 ms (see //unkeybeepgap to change it)
```

Test the GPIO lines without wiring

```
> //remotegpio1,on
Remote GPIO output 1 forced ON (sent to the peer; local input ignored until
//gpiorelease)
> //gpiorelease
All GPIO forcing released -- inputs and outputs back to normal pass-through.
```

Output 1 at the far site switches on until the force is released.

Track down intermittent audio dropouts

Turn on the fault log at both sites and leave the link in normal use:

```
> //faultlog,on
Fault logging: ON (//faultlog to view, //faultlog,clear to empty)
```

When a dropout is heard, print the log on the module whose audio dropped:

```
> //faultlog
--- Fault log: 3 record(s), oldest first, logging ON, now @184211ms ---
     52310ms  LOOP STALL   180ms, slowest: serviceDcnBus
     52362ms  DAC DRY      ~150ms of silence (estimate)
     97004ms  LOST FRAME   x4 over 30ms  seq 9631
--- end of fault log ---
```

Times are milliseconds since start-up. Here a busy moment on the module itself caused the first dropout, and four packets lost in transit caused the second. Fault types are listed under Troubleshooting.

Front panel

Three LEDs show the link at a glance, the display steps through eleven status pages, and the mode button changes pages or, held at power-up, starts Wi-Fi setup.

LEDs

LED	On solid	Blinking	Off
Status	Ethernet link up	Fast (~5 Hz): no Ethernet link. Slow (1 Hz): Wi-Fi setup mode	
Link	Heartbeats arriving from the peer		Peer not heard for 3 s
Activity		Fast: audio or DCN data moving, either direction	Idle, or peer not reachable

A healthy idle pair shows Status and Link on solid and Activity off.

Mode button

Action	Result
Press during normal operation	Next display page
Hold while powering up, for 2 s	Wi-Fi setup mode: the display shows a QR code to join from a phone (see Getting started)
Press during Wi-Fi setup mode	Leaves setup mode without changes and restarts

Display pages

The module shows a splash screen with the firmware version for 2 s at start-up, then the Summary page. Each press of the mode button moves to the next page; after the last it returns to Summary.

Page	Shows
Summary	One line each for Ethernet and peer, audio, DCN and GPIO (codes below)
Link Check	Heartbeat, Audio, GPIO and Serial, each with a tick or cross for OUT (the remote receives it) and IN (this module receives it), and the most likely fault in words: for example "Remote can't hear us", "Remote address changed", "All working"
Link	Ethernet up, down or no gateway; this module's IP address; peer heartbeat
Audio	PTT, audio being sent, COR
Levels	Input and output peak level as bars from -40 to 0 dBFS, with CLIP when samples hit full scale; the current //rxlevel and //txlevel
Serial	DCN connection, password accepted, data moving
DCN Monitor	The last characters sent (S>N) and received (N>S) on the DCN bridge
GPIO	State of the two inputs and two outputs

Page	Shows
Data Rate	Meters with needles for data out and in (log scale, 1 kbps to 1 Mbps), the total rate, and the data volume since start-up; all at IP level, every stream included
Scope In	Live waveform of the local input audio, before filtering
Scope Out	Live waveform of the audio arriving from the peer

On the Summary page:

- `Eth:UP Pr:OK`: Ethernet link and peer heartbeat. `DOWN` and `LOST` when not; `Eth:NO GW` when the gateway or peer doesn't answer.
- `TX:ACT RX:ACT PCM`: sending audio, receiving audio (COR), and the codec the peer is sending (`--` before any audio has arrived).
- `DCN:OK Data:ACT`: DCN connected with the password accepted, and data moving.
- `I1:A I2:- O1:1 O2:0`: inputs active (`A`) or idle (`-`), outputs on (`1`) or off (`0`).

The display updates only the parts of the screen that change, so it no longer interrupts audio. The scope, Link Check, Levels and Data Rate pages redraw once a second, so they never compete with audio; `//scopegain` makes the trace larger or smaller. Modules with the 16x2 character display show every page except the two scope pages.

Troubleshooting

The quickest route is `//linktest` on each module: it checks everything in order and names the first thing to fix. The Link Check display page shows the same at a glance without a laptop. If you need help, `//report` prints everything support needs in one block to copy into an email, with the link password hidden.

Start with `//status` on both modules: it shows which item isn't working and why. `//status,detail` shows each stream in each direction and which router to check, `//net` shows every setting, and `//console,on` shows connection messages (`[DCN]`, `[LINK]`) as they happen.

Symptom	Likely cause	What to do
`Ethernet: connected, but the gateway doesn't answer`; display shows `NO GW`	This module's IP address, netmask or gateway is wrong for the site	Check them against the router with `//net`, or run `//setup`
`Remote device: NOT connected`	Wrong remote address, the peer is offline, ports not forwarded at this site, or this site is behind carrier-grade NAT	Check the remote address points at the other site; forward the UDP ports here; check the router's WAN address isn't 100.64.x.x to 100.127.x.x
Audio or GPIO `NOT working` in one direction	That port isn't forwarded on one router	`//status,detail` names the port and the site
Remote IP address `not resolved yet`	Name misspelled, or no working DNS server	Check the name; set a server with `//dns,8.8.8.8`
`Serial channel: NOT working -- not connected`	TCP 7303 not reachable at either site, or a UPnP port conflict on the router	Forward TCP 7303 at one site or both; `//connect` retries at once
Serial connects then drops; `peer password REJECTED`	Passwords differ	Set the same `//setpassword` on both; it is case-sensitive
Serial working, but bus devices don't respond	Baud rate mismatch	Set `//dcnbaud` to the bus speed on both modules
Audio channel working but nothing heard	PTT not reaching the module, or levels too low	Watch `PTT: ACTIVE` on the Audio page; send `//tone,1000,3000` from the far site
Audio too quiet after setting `pre` on a path	Pre-emphasis lowers 1 kHz by about 9 dB	Raise `//rxlevel` (input) or `//txlevel` (output) by about 14
Hum or buzz at 67-254 Hz in radio audio	CTCSS from the discriminator passing through	`//rxhpf,on`

Symptom	Likely cause	What to do
Muffled or harsh audio	Emphasis set on a path whose equipment already handles it, or missing where it doesn't	Check `//filters` against the equipment at each end (see Connecting to radios)
Choppy audio over the internet	Not enough upload bandwidth, or packet loss	Compare `//bitrate` (IP column) with the upload speed; try `//codec,adpcm,` or `//codec,opus` with `//opusbitrate,16000` (see Choosing a codec); use the fault log
Short dropouts when the module is busy	Brief stalls at the module itself	Check `Loop worst` in `//net`; `//dacprime,5` adds 20 ms of tolerance and 20 ms of delay
Two modules show the same MAC address	Address clash on the same network	Give one module another MAC with `//mac`, then `//reboot`
COR stays on after the far site unkeys	End marker lost; COR falls back to the timeouts	Normal for up to `//audiotimeout` + `//pttdelay` (800 ms by default)
Not sure whether a site has internet	Router, ISP or local settings	//ping on that module: its result line says whether the gateway, the internet or neither answers
//bitrate shows 0.0 out, even for the heartbeat	Nothing is leaving the module: no remote address, Ethernet down, or the gateway doesn't answer	//status; fix the gateway and netmask with //gateway and //netmask, then //reboot
DCN frames to the module get no reply	No address set, a different address, or the frame isn't /FFTT:...:XX with CR LF	//setaddr shows the address; //net shows how many commands were handled and refused
A DCN command replies ERR,NOT ALLOWED	The command isn't on the DCN allowed list	Use the USB console for setup, network, password and reboot commands

Symptom	Likely cause	What to do
//status: WARNING: the remote's packets come from X, but this board sends to Y; Link Check page: Remote address changed	The far site's public address changed, or its dynamic DNS name is out of date	Update //remoteip with the address shown, or fix the far site's DNS updater. If the far site can dial in on the serial link, the module follows it by itself until a reboot
Boot summary or //status: Started by WATCHDOG or CRASH	The firmware stopped responding for 60 s, or hit a fault, and restarted itself	Send //report to support; it includes the count of unexpected restarts
Started by BROWNOUT	The supply voltage dipped	Check the 10-15 V supply, its current rating and the power wiring
//net: Jitter buffer raised, or near 100 ms	Network jitter larger than the buffer caused repeated short dropouts	Normal on a busy internet path. //looptest shows the jitter; if dropouts continue at 100 ms, the path itself is losing packets
//looptest: Refused, or Opus stays at 10 ms packets	The remote isn't heard, or runs older firmware	Update both modules to v85; //status,detail shows the remote's firmware version
Levels page shows CLIP	Audio is hitting full scale	Input: lower //inputlevel or //rxlevel. Output: lower //rxlevel at the remote

Fault log entries

With `//faultlog,on`, the module records each audio fault and link event. Repeats of the same fault within 250 ms are counted in one record (`x4 over 30ms`). The log keeps the latest 128 records in memory and is cleared by a reboot.

Entry	Meaning
`UNDERRUN`	Audio from the peer paused mid-transmission; shows the gap. Usually a stall at the sending site or on the network

Entry	Meaning
`LOST FRAME`	A packet never arrived and silence was played in its place
`LATE RESYNC`	Several packets arrived too late to play; the buffer realigned to the stream
`JUMP RESYNC`	The packet sequence jumped, typically after an outage or a peer reboot
`TRIM`	One 10 ms frame was dropped to remove extra delay
`I2S SHORT WR`	The audio output could not accept a full frame at once
`DAC DRY`	The audio output ran out of queued audio; shows roughly how long it played silence
`LOOP STALL`	The module was busy for over 40 ms; names the slowest task
`GATEWAY DOWN` / `GATEWAY OK`	The gateway (or peer, on the same network) stopped or resumed answering
`ETH DOWN` / `ETH UP`	The Ethernet cable or link went down or came back
`PEER LOST` / `PEER OK`	Heartbeats from the peer stopped or resumed
`DNS FAIL`	Looking up the remote name failed
`PEER ADDR`	The remote name now points to a new address
`DHCP`	A DHCP lease renewal or rebind took place

Re-keying PTT within about 1 s of releasing it can also appear as an `UNDERRUN`. Printing the log briefly pauses the module, so it may record its own `LOOP STALL`.